

CO5-AT1

Name : Tharish K

CSA1410

Reg no : 192412004

Edge AI System for Industrial Quality Control

1. System Design

The proposed system combines **Edge AI, IoT sensors/cameras, and a PySpark-based backend** to perform real-time product inspection on a high-speed manufacturing line.

Basic architecture:

Production Line



Industrial Camera + Sensors



Edge AI Device



| Image Capture |

| Preprocessing |

| AI Detection |

| Agent Decision |



Defect / No Defect



| Central PySpark Backend |

| RDD Batch Processing |

| Historical Defect Analysis |

| Model Retraining |



Updated Model + Thresholds



Edge AI Device

2. Edge AI-Based Inspection

An industrial camera continuously captures product images while products move through the manufacturing line. The images are processed locally by an **Edge AI device** such as an NVIDIA Jetson or industrial computer.

The AI model classifies each product as:

- **Good product**
- **Defective product**
- **Defect type and severity**

Because processing occurs at the edge, the system can make decisions within milliseconds without sending every image to the cloud. A defective product can therefore be immediately removed using an actuator or robotic mechanism.

3. PySpark Backend and RDD Operations

The central backend stores historical inspection data such as:

- Product ID
- Production batch
- Defect type
- Defect frequency
- Temperature and humidity
- Machine operating conditions
- Inspection confidence
- Timestamp

PySpark **RDD (Resilient Distributed Dataset)** operations can process this large historical dataset.

Example operations:

```
defects = sc.textFile("defect_data.csv")
```

```
records = defects.map(lambda x: x.split(","))
```

```
defect_data = records.filter(lambda x: x[3] == "DEFECT")
```

```
defect_count = defect_data.map(
```

```
    lambda x: (x[2], 1)
```

```
).reduceByKey(lambda a, b: a + b)
```

```
print(defect_count.collect())
```

Here, `map()` transforms the data, `filter()` selects defective products, and `reduceByKey()` calculates the number of defects for each defect category.

4. Distributed Processing for Model Retraining

Manufacturing plants can generate **millions of product images and inspection records**. Processing this data on a single computer would be slow.

PySpark distributes the historical data across multiple worker nodes. Each node processes a portion of the dataset simultaneously.

The retraining process can be:

1. Collect historical defect data from edge devices.
2. Store the data in distributed storage.
3. Use PySpark RDDs to clean and analyze the data.
4. Identify frequently occurring and newly appearing defects.
5. Prepare a new training dataset.
6. Retrain or fine-tune the AI model using the processed data.
7. Validate the updated model.
8. Deploy the improved model to edge devices.

Thus, **distributed processing reduces training time and allows the system to handle large-scale industrial datasets efficiently**.

5. Intelligent Agent Collaboration

An intelligent software agent operates at each edge device. It continuously monitors the production environment and makes local inspection decisions.

The **Edge Agent** performs:

- Real-time image analysis
- Defect detection
- Confidence evaluation
- Threshold adjustment
- Local anomaly detection
- Immediate rejection of defective products

The **Central Agent** performs:

- Historical data analysis

- Detection of long-term defect trends
- Model retraining
- Comparison of production lines
- Generation of updated inspection policies

The two agents communicate through a secure network. The edge agent sends important events and summarized data rather than every raw image, reducing network traffic.

6. Adaptive Inspection Threshold

Production conditions can change due to factors such as **machine vibration, lighting, temperature, material variation, or tool wear**.

For example, if the AI normally uses a defect confidence threshold of **0.80**, changing production conditions may increase false detections. The central system can analyze historical performance and recommend a new threshold, such as **0.85**.

The edge agent receives the updated threshold and applies it locally.

Normal condition → Threshold = 0.80

Changed condition → Central analysis

↓

New threshold = 0.85

↓

Edge AI update

This allows the system to **adapt automatically without completely stopping the production line**.

7. Advantages

- **Real-time detection:** Defects are detected immediately at the edge.
- **Low latency:** No need to send every image to a remote server.
- **Scalability:** PySpark distributes large historical datasets across multiple machines.
- **Continuous improvement:** Historical defect data supports regular model retraining.
- **Adaptive operation:** Inspection thresholds can change according to production conditions.
- **Reduced bandwidth:** Only important results and selected data are transmitted.
- **Higher reliability:** Local inspection can continue even if the central connection is temporarily unavailable.

8. Conclusion

The proposed **Edge AI + PySpark quality-control system** combines fast local decision-making with powerful centralized data processing. Edge intelligent agents detect product defects in real time, while the PySpark backend uses RDD-based distributed processing to analyze large historical datasets and support model retraining. The central and edge agents continuously exchange models, defect

statistics, and inspection thresholds, allowing the system to adapt to changing production conditions. Therefore, the design provides a **scalable, low-latency, intelligent, and continuously improving solution for high-speed industrial quality control**.